



API Liga Deportiva

Sports League API

Fase 6: Standings y Estadísticas

Conceptos Clave

LINQ Avanzado | GroupBy | Datos Calculados (no almacenados)
DTOs de Reporte | Tabla de Posiciones | Goleadores | Ranking de Tarjetas

Guía paso a paso para construcción en clase

1. Objetivos de la Fase 6

Al finalizar esta fase, los estudiantes serán capaces de:

- Calcular datos dinámicamente con LINQ en lugar de almacenarlos en la base de datos
- Usar GroupBy, Sum, Count y OrderByDescending para generar reportes
- Crear DTOs específicos para estadísticas y reportes (no ligados a entidades)
- Implementar la tabla de posiciones de un torneo con puntos, goles y diferencia de gol
- Crear la tabla de goleadores y ranking de tarjetas por torneo
- Diseñar un StandingsService como servicio de solo lectura (sin escritura)

2. Conceptos Nuevos en esta Fase

2.1 Datos calculados vs almacenados

La tabla de posiciones NO se almacena en la base de datos. Se calcula en tiempo real a partir de los MatchResults existentes. Esto tiene ventajas importantes:

- ✓ Calculado (nuestro enfoque):
 - Siempre está actualizado automáticamente
 - No hay riesgo de inconsistencia
 - No necesita sincronización al registrar resultados
- X Almacenado (alternativa):
 - Requiere actualizar la tabla cada vez que se registra un resultado
 - Puede quedar desincronizado si falla una actualización
 - Más rápido en lectura pero más complejo de mantener

Enseñanza clave

Para una liga con pocos equipos (8-20), calcular en tiempo real es perfectamente eficiente. Solo se necesitaría almacenar si tuviéramos miles de partidos y requiriéramos respuestas en milisegundos. En el mundo real, ligas como la Premier League calculan en tiempo real.

2.2 LINQ avanzado

Esta fase usa operaciones LINQ que van más allá del Where y Select básico:

Operación	Uso
GroupBy	Agrupar goles por jugador para contar cuántos anotó cada uno
Sum	Sumar goles a favor, goles en contra
Count	Contar partidos jugados, ganados, empatados, perdidos
SelectMany	Aplanar colecciones anidadas (todos los goles de todos los partidos)
OrderByDescending	Ordenar tabla por puntos, diferencia de gol, goles a favor
ThenByDescending	Segundo y tercer criterio de desempate
Concat	Unir resultados como local y como visitante para un mismo equipo

2.3 Reglas de puntuación

Victoria = 3 puntos
 Empate = 1 punto
 Derrota = 0 puntos

Criterios de desempate (en orden):

1. Puntos (mayor)
2. Diferencia de gol (mayor)
3. Goles a favor (mayor)

3. Crear los DTOs de Estadísticas

Estos DTOs no corresponden a ninguna entidad de la BD. Son estructuras de solo lectura para reportes:

3.1 StandingDTO (tabla de posiciones)

SportsLeague.API/DTOs/Response/StandingDTO.cs

```
namespace SportsLeague.API.DTOs.Response;

public class StandingDTO
{
    public int Position { get; set; }
    public int TeamId { get; set; }
    public string TeamName { get; set; } = string.Empty;
    public int MatchesPlayed { get; set; } // PJ
    public int Wins { get; set; } // PG
    public int Draws { get; set; } // PE
}
```

```

    public int Losses { get; set; }           // PP
    public int GoalsFor { get; set; }         // GF
    public int GoalsAgainst { get; set; }     // GC
    public int GoalDifference { get; set; }    // DG
    public int Points { get; set; }           // Pts
}

```

3.2 TopScorerDTO (goleadores)

SportsLeague.API/DTOs/Response/TopScorerDTO.cs

```

namespace SportsLeague.API.DTOs.Response;

public class TopScorerDTO
{
    public int PlayerId { get; set; }
    public string PlayerName { get; set; } = string.Empty;
    public string TeamName { get; set; } = string.Empty;
    public int Goals { get; set; }
    public int Penalties { get; set; }
    public int MatchesWithGoals { get; set; }
}

```

3.3 CardStatsDTO (tarjetas)

SportsLeague.API/DTOs/Response/CardStatsDTO.cs

```

namespace SportsLeague.API.DTOs.Response;

public class CardStatsDTO
{
    public int PlayerId { get; set; }
    public string PlayerName { get; set; } = string.Empty;
    public string TeamName { get; set; } = string.Empty;
    public int YellowCards { get; set; }
    public int RedCards { get; set; }
    public int TotalCards { get; set; }
}

```

DTOs sin Entity

Estos DTOs no tienen CreateMap en AutoMapper porque no se mapean desde una entidad. Se construyen manualmente en el Service con LINQ. Son DTOs de reporte/estadística.

4. Definir la Interface IStandingsService

SportsLeague.Domain/Interfaces/Services/IStandingsService.cs

```
namespace SportsLeague.Domain.Interfaces.Services;

public interface IStandingsService
{
    Task<object> GetStandingsAsync(int tournamentId);
    Task<object> GetTopScorersAsync(int tournamentId);
    Task<object> GetCardStatsAsync(int tournamentId);
}
```

💡 ¿Por qué retorna object?

El Domain no conoce los DTOs de la capa API. Usamos object como tipo de retorno genérico. Otra opción sería crear clases de resultado en Domain, pero para simplificar usamos object. En la práctica, el Service construirá objetos anónimos o listas de objetos anónimos que el Controller enviará como JSON.

5. Implementar el StandingsService

Este es el Service más interesante del proyecto desde el punto de vista de LINQ. No escribe datos, solo los calcula:

SportsLeague.Domain/Services/StandingsService.cs

```
using Microsoft.Extensions.Logging;
using SportsLeague.Domain.Entities;
using SportsLeague.Domain.Enums;
using SportsLeague.Domain.Interfaces.Repositories;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.Domain.Services;

public class StandingsService : IStandingsService
{
    private readonly ITournamentRepository _tournamentRepository;
    private readonly ITournamentTeamRepository _tournamentTeamRepository;
    private readonly IMatchRepository _matchRepository;
    private readonly IMatchResultRepository _matchResultRepository;
    private readonly IGoalRepository _goalRepository;
    private readonly ICardRepository _cardRepository;
    private readonly ILogger<StandingsService> _logger;

    public StandingsService(
        ITournamentRepository tournamentRepository,
        ITournamentTeamRepository tournamentTeamRepository,
        IMatchRepository matchRepository,
        IMatchResultRepository matchResultRepository,
```

```
        IGoalRepository goalRepository,  
        ICardRepository cardRepository,  
        ILogger<StandingsService> logger)  
    {  
        _tournamentRepository = tournamentRepository;  
        _tournamentTeamRepository = tournamentTeamRepository;  
        _matchRepository = matchRepository;  
        _matchResultRepository = matchResultRepository;  
        _goalRepository = goalRepository;  
        _cardRepository = cardRepository;  
        _logger = logger;  
    }
```

GetStandingsAsync - Tabla de posiciones

```
public async Task<object> GetStandingsAsync(int tournamentId)  
{  
    var tournament = await _tournamentRepository.GetByIdAsync(tournamentId);  
    if (tournament == null)  
        throw new KeyNotFoundException(  
            $"No se encontró el torneo con ID {tournamentId}");  
  
    // Obtener equipos inscritos  
    var tournamentTeams = await _tournamentTeamRepository  
        .GetByTournamentAsync(tournamentId);  
  
    // Obtener partidos finalizados del torneo  
    var matches = (await _matchRepository  
        .GetByTournamentAsync(tournamentId))  
        .Where(m => m.Status == MatchStatus.Finished)  
        .ToList();  
  
    // Obtener resultados de esos partidos  
    var matchIds = matches.Select(m => m.Id).ToHashSet();  
    var allResults = new List<MatchResult>();  
    foreach (var matchId in matchIds)  
    {  
        var result = await _matchResultRepository.GetByMatchIdAsync(matchId);  
        if (result != null) allResults.Add(result);  
    }  
  
    // Construir diccionario matchId -> result para acceso rápido  
    var resultsByMatch = allResults.ToDictionary(r => r.MatchId);  
  
    // Calcular standings para cada equipo  
    var standings = tournamentTeams.Select(tt =>  
    {  
        var teamId = tt.TeamId;  
        var teamName = tt.Team.Name;  
  
        // Partidos como local con resultado  
        var homeMatches = matches
```

```

        .Where(m => m.HomeTeamId == teamId &&
resultsByMatch.ContainsKey(m.Id))
        .Select(m => resultsByMatch[m.Id])
        .ToList();

    // Partidos como visitante con resultado
    var awayMatches = matches
        .Where(m => m.AwayTeamId == teamId &&
resultsByMatch.ContainsKey(m.Id))
        .Select(m => resultsByMatch[m.Id])
        .ToList();

    // Calcular estadísticas
    int homeWins = homeMatches.Count(r => r.HomeGoals > r.AwayGoals);
    int homeDraws = homeMatches.Count(r => r.HomeGoals == r.AwayGoals);
    int homeLosses = homeMatches.Count(r => r.HomeGoals < r.AwayGoals);
    int homeGF = homeMatches.Sum(r => r.HomeGoals);
    int homeGC = homeMatches.Sum(r => r.AwayGoals);

    int awayWins = awayMatches.Count(r => r.AwayGoals > r.HomeGoals);
    int awayDraws = awayMatches.Count(r => r.AwayGoals == r.HomeGoals);
    int awayLosses = awayMatches.Count(r => r.AwayGoals < r.HomeGoals);
    int awayGF = awayMatches.Sum(r => r.AwayGoals);
    int awayGC = awayMatches.Sum(r => r.HomeGoals);

    int totalWins = homeWins + awayWins;
    int totalDraws = homeDraws + awayDraws;
    int totalLosses = homeLosses + awayLosses;
    int totalGF = homeGF + awayGF;
    int totalGC = homeGC + awayGC;

    return new
    {
        TeamId = teamId,
        TeamName = teamName,
        MatchesPlayed = homeMatches.Count + awayMatches.Count,
        Wins = totalWins,
        Draws = totalDraws,
        Losses = totalLosses,
        GoalsFor = totalGF,
        GoalsAgainst = totalGC,
        GoalDifference = totalGF - totalGC,
        Points = (totalWins * 3) + totalDraws
    };
})
.OrderByDescending(s => s.Points)
.ThenByDescending(s => s.GoalDifference)
.ThenByDescending(s => s.GoalsFor)
.Select((s, index) => new
{
    Position = index + 1,
    s.TeamId,
    s.TeamName,
    s.MatchesPlayed,
    s.Wins,
    s.Draws,

```

```

        s.Losses,
        s.GoalsFor,
        s.GoalsAgainst,
        s.GoalDifference,
        s.Points
    })
    .ToList();

    _logger.LogInformation(
        "Generated standings for tournament {TournamentId}", tournamentId);
    return standings;
}

```

💡 Desglose del algoritmo

1) Obtener equipos del torneo. 2) Obtener partidos Finished con resultados. 3) Para cada equipo, separar partidos como local y visitante. 4) Calcular W/D/L/GF/GC sumando local + visitante. 5) Puntos = $(W \times 3) + D$. 6) Ordenar por Puntos → DG → GF. 7) Asignar posición con `Select((s, index))`.

GetTopScorersAsync - Tabla de goleadores

```

public async Task<object> GetTopScorersAsync(int tournamentId)
{
    var tournament = await _tournamentRepository.GetByIdAsync(tournamentId);
    if (tournament == null)
        throw new KeyNotFoundException(
            $"No se encontró el torneo con ID {tournamentId}");

    // Obtener todos los partidos del torneo
    var matches = await _matchRepository.GetByTournamentAsync(tournamentId);
    var matchIds = matches.Select(m => m.Id).ToHashSet();

    // Obtener goles de esos partidos
    var allGoals = new List<Goal>();
    foreach (var matchId in matchIds)
    {
        var goals = await _goalRepository.GetByMatchWithDetailsAsync(matchId);
        allGoals.AddRange(goals);
    }

    // Excluir autogoles del conteo de goleadores
    var scorerGoals = allGoals
        .Where(g => g.Type != Enums.GoalType.OwnGoal)
        .ToList();

    // Agrupar por jugador
    var topScorers = scorerGoals
        .GroupBy(g => new { g.PlayerId, g.Player.FirstName, g.Player.LastName,
            g.Player.TeamId })
        .Select(group => new
        {
            PlayerId = group.Key.PlayerId,

```

```

        PlayerName = group.Key.FirstName + " " + group.Key.LastName,
        TeamName = group.First().Player.Team?.Name ?? "N/A",
        Goals = group.Count(),
        Penalties = group.Count(g => g.Type == Enums.GoalType.Penalty),
        MatchesWithGoals = group.Select(g => g.MatchId).Distinct().Count()
    })
    .OrderByDescending(s => s.Goals)
    .ThenByDescending(s => s.MatchesWithGoals)
    .ToList();

_logger.LogInformation(
    "Generated top scorers for tournament {TournamentId}", tournamentId);
return topScorers;
}

```

Excluir autogoles

Los OwnGoal no cuentan para la tabla de goleadores (son goles en contra). También mostramos cuántos fueron de penal y en cuántos partidos distintos anotó el jugador.

GetCardStatsAsync - Ranking de tarjetas

```

public async Task<object> GetCardStatsAsync(int tournamentId)
{
    var tournament = await _tournamentRepository.GetByIdAsync(tournamentId);
    if (tournament == null)
        throw new KeyNotFoundException(
            $"No se encontró el torneo con ID {tournamentId}");

    var matches = await _matchRepository.GetByTournamentAsync(tournamentId);
    var matchIds = matches.Select(m => m.Id).ToHashSet();

    var allCards = new List<Card>();
    foreach (var matchId in matchIds)
    {
        var cards = await _cardRepository.GetByMatchWithDetailsAsync(matchId);
        allCards.AddRange(cards);
    }

    var cardStats = allCards
        .GroupBy(c => new { c.PlayerId, c.Player.FirstName, c.Player.LastName })
        .Select(group => new
        {
            PlayerId = group.Key.PlayerId,
            PlayerName = group.Key.FirstName + " " + group.Key.LastName,
            TeamName = group.First().Player.Team?.Name ?? "N/A",
            YellowCards = group.Count(c => c.Type == Enums.CardType.Yellow),
            RedCards = group.Count(c => c.Type == Enums.CardType.Red),
            TotalCards = group.Count()
        })
        .OrderByDescending(s => s.RedCards)
        .ThenByDescending(s => s.YellowCards)

```

```
        .ToList();

        _logger.LogInformation(
            "Generated card stats for tournament {TournamentId}", tournamentId);
        return cardStats;
    }
}
```

6. Crear el StandingsController

SportsLeague.API/Controllers/StandingsController.cs

```
using Microsoft.AspNetCore.Mvc;
using SportsLeague.Domain.Interfaces.Services;

namespace SportsLeague.API.Controllers;

[ApiController]
[Route("api")]
public class StandingsController : ControllerBase
{
    private readonly IStandingsService _standingsService;

    public StandingsController(IStandingsService standingsService)
    {
        _standingsService = standingsService;
    }

    [HttpGet("standings")]
    public async Task<ActionResult> GetStandings([FromQuery] int tournamentId)
    {
        try
        {
            var standings = await _standingsService.GetStandingsAsync(tournamentId);
            return Ok(standings);
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
    }

    [HttpGet("stats/scorers")]
    public async Task<ActionResult> GetTopScorers([FromQuery] int tournamentId)
    {
        try
        {
            var scorers = await _standingsService.GetTopScorersAsync(tournamentId);
            return Ok(scorers);
        }
        catch (KeyNotFoundException ex)
        {
        }
    }
}
```

```

        return NotFound(new { message = ex.Message });
    }
}

[HttpGet("stats/cards")]
public async Task<ActionResult> GetCardStats([FromQuery] int tournamentId)
{
    try
    {
        var cards = await _standingsService.GetCardStatsAsync(tournamentId);
        return Ok(cards);
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
}
}

```

💡 [FromQuery] y rutas

[FromQuery] int tournamentId toma el parámetro de la URL: GET /api/standings?tournamentId=1. La ruta base [Route("api")] permite tener endpoints como /api/standings y /api/stats/scorers sin un nombre de controller en la URL.

7. Actualizar Program.cs

```

// — Services (agregar) —
builder.Services.AddScoped<IStandingsService, StandingsService>();

```

📄 Sin nuevos Repositories

Esta fase no agrega nuevos Repositories ni migraciones. StandingsService reutiliza los repositorios existentes para calcular las estadísticas.

8. Probar la API

Para probar las estadísticas necesitamos datos completos. Seguir estos pasos:

Preparación de datos de prueba

1. Crear 3-4 equipos con jugadores (2-3 jugadores por equipo)
2. Crear un torneo e inscribir todos los equipos

3. Iniciar el torneo (PATCH status → InProgress)
4. Crear 4-6 partidos entre los equipos
5. Para cada partido:
 - a. PATCH status → InProgress
 - b. Registrar goles (POST /api/match/{id}/goals)
 - c. Registrar tarjetas (POST /api/match/{id}/cards)
 - d. PATCH status → Finished
 - e. Registrar resultado (POST /api/match/{id}/result)

Consultar tabla de posiciones

```
GET /api/standings?tournamentId=1
```

Respuesta esperada (ejemplo):

```
[
  {
    "position": 1,
    "teamName": "Atlético Nacional",
    "matchesPlayed": 3,
    "wins": 2, "draws": 1, "losses": 0,
    "goalsFor": 5, "goalsAgainst": 2,
    "goalDifference": 3,
    "points": 7
  },
  {
    "position": 2,
    "teamName": "Independiente Medellín",
    "matchesPlayed": 3,
    "wins": 1, "draws": 1, "losses": 1,
    "goalsFor": 4, "goalsAgainst": 3,
    "goalDifference": 1,
    "points": 4
  }
]
```

Consultar goleadores

```
GET /api/stats/scorers?tournamentId=1
```

Consultar tarjetas

```
GET /api/stats/cards?tournamentId=1
```

Verificaciones esperadas

Acción	Cód.	Resultado
GET /api/standings?tournamentId=1	200	Tabla ordenada por puntos
GET /api/standings?tournamentId=999	404	Torneo no encontrado
GET standings sin partidos terminados	200	Tabla con todos los equipos en 0
GET standings después de registrar resultado	200	Tabla actualizada automáticamente
GET /api/stats/scorers?tournamentId=1	200	Goleadores ordenados por goles
Verificar que OwnGoals no cuentan	200	Autogoles excluidos del ranking
GET /api/stats/cards?tournamentId=1	200	Tarjetas ordenadas por rojas
Empate en puntos	200	Desempate por DG, luego GF

9. Archivos Nuevos y Modificados

Archivo	Estado	Capa
API/DTOs/Response/StandingDTO.cs	NUEVO	API
API/DTOs/Response/TopScorerDTO.cs	NUEVO	API
API/DTOs/Response/CardStatsDTO.cs	NUEVO	API
API/Controllers/StandingsController.cs	NUEVO	API
Domain/Interfaces/Services/IStandingsService.cs	NUEVO	Domain
Domain/Services/StandingsService.cs	NUEVO	Domain
API/Program.cs	Actualizado	API

Sin migraciones

Esta fase no requiere migraciones porque no se crea ninguna tabla nueva. Las estadísticas se calculan a partir de datos existentes.

10. Resumen de Conceptos Aprendidos

Concepto	Aplicación en esta Fase
Datos calculados	Standings calculados en tiempo real, no almacenados

Concepto	Aplicación en esta Fase
LINQ GroupBy	Agrupar goles y tarjetas por jugador
LINQ Sum / Count	Sumar goles, contar partidos ganados/empatados/perdidos
OrderByDescending + ThenBy	Desempate por puntos → DG → GF
Select con index	Asignar posición automática: Select((s, index))
Objetos anónimos	new { TeamId, Points, ... } sin clase formal
DTOs de reporte	DTOs que no mapean a entidades sino a datos calculados
Service de solo lectura	StandingsService solo consulta, nunca escribe
[FromQuery]	Parámetros de URL: ?tournamentId=1
Sin migraciones	Reutilizar datos existentes para nuevas funcionalidades

11. Próxima Fase

En la Fase 7 agregaremos:

- Middleware global de manejo de excepciones (reemplazar try/catch en cada Controller)
- Paginación en los endpoints de listado (PagedResult<T>)
- Filtros de búsqueda con query parameters (?city=Medellín&status=1)
- Modelo de respuesta estándar (ApiResponse<T>) para consistencia en toda la API